



UNIVERSIDAD AUTÓNOMA DE CHIHUAHUA

Facultad de Ingeniería

Ingeniería en Ciencias de la Computación

Cómputo Paralelo y Distribuido

Docente: José Saúl De Lira Miramontes

Grupo: 8CC2

Alumno:

ABEL GONZALEZ MIRELES 361031

ADRIAN CALEB JARAMILLO FLORES 367857

MIGUEL DAVID RODRIGUEZ GONZALEZ 343786

Chihuahua, Chih., Mayo 2026

Indice

Indice	2
1. Descripción General del Proyecto	3
2. Arquitectura del Sistema	3
2.1 Flujo de Trabajo	3
2.2 Topología del Clúster	4
2.3 Estructura de Directorios	4
3. Componentes Principales	4
3.1 ETL (src/etl.py)	4
Columnas de entrada relevantes	4
Columnas derivadas calculadas	5
Funciones principales	5
3.2 Análisis Distribuido (src/analisis.py)	5
3.3 Dashboard (dashboard/app.py)	6
4. Configuración Docker	6
4.1 Imágenes	6
4.2 Red y Volúmenes	6
5. Guía de Uso	7
5.1 Requisitos Previos	7
5.2 Pasos de Ejecución	7
5.3 URLs de Acceso	7
5.4 Escalado de Workers	8
Anexo — Imágenes del Sistema en Funcionamiento	8
A.1 Pipeline ETL	8
A.2 Análisis Distribuido con Ray	8
A.3 Dashboard Streamlit	8
7. Dependencias y Tecnologías	9
8. Consideraciones de Diseño	9
8.1 Encoding de los Datos INEGI	9
8.2 Estrategia ETL vs. Análisis	9
8.3 Formato Parquet	10
8.4 Índice de Gravedad	10

1. Descripción General del Proyecto

ATUS Mexico es un sistema de computo distribuido diseñado para procesar y analizar los datos historicos de accidentes viales del INEGI (Anuario Estadistico de Accidentes de Transito Urbano y Suburbano) correspondientes al periodo 1997-2024. El sistema ejecuta pipelines de limpieza, transformacion y analisis sobre un cluster Ray, y expone los resultados a traves de un dashboard interactivo construido con Streamlit y Plotly.

El objetivo central del proyecto es demostrar las ventajas del procesamiento distribuido sobre el procesamiento secuencial mediante un benchmark medible, ofreciendo al mismo tiempo una herramienta de analisis exploratorio de accidentes viales en Mexico.

Aspecto	Detalle
Nombre del sistema	ATUS Mexico -- Analisis Distribuido de Accidentes Viales
Fuente de datos	INEGI -- ATUS 1997 a 2024 (archivos CSV anuales)
Motor distribuido	Ray 2.10
Dashboard	Streamlit 1.33 + Plotly 5.21
Orquestacion	Docker Compose
Lenguaje	Python 3.11
Formato intermedio	Apache Parquet (compresion Snappy)

2. Arquitectura del Sistema

2.1 Flujo de Trabajo

El pipeline principal sigue la siguiente secuencia de etapas:

CSVs raw -> ETL (Ray) -> Parquet -> Analisis (Ray) -> JSON -> Dashboard

- CSVs raw: archivos anuales del ATUS descargados del portal del INEGI, nombrados como atus_anual_AAAA.csv y colocados en data/raw/.
- ETL: limpieza de columnas, correccion de encoding latin-1, normalizacion de tipos numericos, calculo de columnas derivadas (TOTAL_MUERTOS, TOTAL_HERIDOS, GRAVEDAD) y exportacion a Parquet con compresion Snappy.
- Analisis distribuido (Ray): lectura de los Parquet y ejecucion paralela de multiples tareas de agregacion por entidad, municipio, hora, mes, causa y tipo de accidente.
- JSON de resultados: los indicadores calculados se serializan en data/reports/resultados.json para ser consumidos por el dashboard.
- Dashboard Streamlit: visualizacion interactiva con cuatro pestanas (Por Estado, Municipios, Temporal, Causas) y pestaña de benchmark Ray vs Pandas.

2.2 Topologia del Cluster

El cluster se compone de tres nodos Docker conectados a traves de una red bridge interna (atus-net):

Servicio	Funcion	Puertos expuestos
ray-head	Head node de Ray + Dashboard Streamlit	8501, 8265, 6379
ray-worker-1	Worker node (2 CPUs, 4 GB RAM)	--
ray-worker-2	Worker node (2 CPUs, 4 GB RAM)	--

El volumen compartido atus-data, montado en /app/data en todos los contenedores, garantiza que los archivos Parquet generados por el ETL sean accesibles por los workers durante el analisis.

2.3 Estructura de Directorios

```
atus_project/
├── src/
│   ├── etl.py           # Limpieza CSV -> Parquet
│   └── analisis.py      # Calculo de indicadores con Ray
├── dashboard/
│   └── app.py           # Dashboard Streamlit
├── docker/
│   ├── Dockerfile       # Head node + dashboard
│   ├── Dockerfile.worker # Worker nodes
│   └── docker-compose.yml # Orquestacion completa
├── data/
│   ├── raw/             # CSVs del ATUS (entrada)
│   ├── parquet/         # Generado por ETL
│   └── reports/         # JSON con resultados del analisis
└── requirements.txt
```

3. Componentes Principales

3.1 ETL (src/etl.py)

Modulo responsable de transformar los archivos CSV crudos del INEGI en archivos Parquet optimizados para consultas analiticas. Lee los datos con encoding latin-1 (estandar oficial del INEGI), selecciona las columnas relevantes y calcula columnas derivadas.

Columnas de entrada seleccionadas

Columna	Descripcion
ID_ENTIDAD	Clave numerica del estado (1-32)
ID_MUNICIPIO	Clave del municipio
ANIO, MES, ID_HORA	Variables temporales del accidente
DIASEMANA	Dia de la semana en texto (Lunes-Domingo)
CAUSAACCI	Causa principal del accidente
TIPACCID	Tipo de accidente
CONDMUERTO / CONDHERIDO	Victimas conductores (fallecidos / heridos)
PASAMUERTO / PASAHERIDO	Victimas pasajeros
PEATMUERTO / PEATHERIDO	Victimas peatones
CICLMUERTO / CICLHERIDO	Victimas ciclistas

Columnas derivadas calculadas

- TOTAL_MUERTOS: suma de todas las columnas de fallecidos.
- TOTAL_HERIDOS: suma de todas las columnas de lesionados.
- GRAVEDAD: indice de severidad = TOTAL_MUERTOS x 3 + TOTAL_HERIDOS.
- NOM_ENTIDAD: nombre textual del estado mapeado desde el catalogo del INEGI.

Funciones principales

- limpiar_csv(path): lee un CSV, aplica correccion de encoding, filtra registros invalidos y calcula las columnas derivadas.
- procesar_archivo_ray(path, salida): tarea Ray remota (@ray.remote) que invoca limpiar_csv y serializa el resultado en Parquet con compresion Snappy.
- etl_secuencial(raw_dir, parquet_dir): procesa los archivos en secuencia en el head node. Es el modo de ejecucion real, ya que los CSVs solo estan montados en el head.
- etl_distribuido(raw_dir, parquet_dir): variante que distribuye tareas entre workers mediante ray.get(futures); disponible para escenarios donde los datos sean accesibles por todos los nodos.

3.2 Analisis Distribuido (src/analisis.py)

Este modulo carga todos los Parquet consolidados en un DataFrame de Pandas, los sube al object store de Ray (ray.put) y distribuye el calculo de indicadores como tareas independientes. Las tareas remotas principales son:

- analisis_por_entidad(df): agrega accidentes, muertos, heridos y gravedad por estado.

- `analisis_por_municipio(df, top_n=50)`: devuelve los 50 municipios con mayor numero de accidentes.
- `analisis_temporal(df)`: genera distribuciones por hora del dia, mes del anio, dia de la semana y anio calendario.
- `analisis_causas(df)`: cuenta accidentes por causa principal y por tipo de accidente.
- `analisis_gravedad(df)`: calcula el resumen global de victimas e identifica los estados con mas accidentes fatales.
- `benchmark(parquet_dir)`: mide y compara el tiempo de procesamiento distribuido (Ray) vs. secuencial (Pandas), calculando el speedup obtenido.

Los resultados de todas las tareas se serializan en `data/reports/resultados.json` para ser consumidos por el dashboard. El benchmark genera un archivo separado `data/reports/benchmark.json`.

3.3 Dashboard (dashboard/app.py)

Interfaz web desarrollada con Streamlit que visualiza los resultados del analisis. Aplica un tema oscuro personalizado (CSS inline) y usa Plotly para todas las graficas. Se estructura en cuatro pestanas:

- Por Estado: barras horizontales de accidentes y fallecidos por estado, mas tabla interactiva completa con todos los indicadores.
- Municipios: top 30 municipios mas afectados con codificacion de color por numero de fallecidos.
- Temporal: linea de tendencia anual 1997-2024, distribucion por hora del dia, por mes y por dia de la semana.
- Causas: grafica de pastel de causas frecuentes, barras de tipos de accidente y tasa de mortalidad por causa.

Los KPIs globales (total de accidentes, fallecidos, heridos, accidentes con fallecidos e indice de gravedad promedio) se muestran como tarjetas en la parte superior del dashboard, fuera de las pestanas.

4. Configuración Docker

4.1 Imágenes

Se usan dos imágenes basadas en python:3.11-slim:

- Dockerfile (head): instala gcc, g++ y libgomp1 para compilar dependencias nativas, luego instala las librerías Python (Ray, Pandas, PyArrow, Streamlit, Plotly, openpyxl, fastparquet). Copia src/ y dashboard/ al contenedor. El comando por defecto arranca Streamlit en el puerto 8501.
- Dockerfile.worker: imagen idéntica en cuanto a dependencias Python, pero sin el código del dashboard. El comando por defecto ejecuta ray start --address=ray-head:6379 --block para unirse al cluster.

4.2 Servicios en docker-compose.yml

Servicio	Descripción
ray-head	Arranca Ray con --head y luego Streamlit. Monta data/raw como solo lectura y comparte atus-data con los workers.
ray-worker-1	Worker conectado al head. Límite de 2 CPUs y 4 GB RAM. Accede a atus-data para leer los Parquet.
ray-worker-2	Idéntico a ray-worker-1; segunda instancia para paralelismo adicional.

4.3 Red y Volúmenes

- atus-net: red bridge dedicada que aísla la comunicación interna del cluster del tráfico externo.
- atus-data: volumen Docker compartido entre todos los servicios; contiene parquet/ y reports/ generados en ejecución.
- data/raw (montaje ro): los CSV del host se montan de solo lectura en el head node para que el ETL pueda leerlos sin copiarlos al contenedor.

5. Guía de Uso

5.1 Requisitos Previos

- Docker Desktop (o Docker Engine) instalado y en ejecución.
- Docker Compose v2 disponible (comando docker compose).

- Archivos CSV del ATUS colocados en `atus_project/data/raw/` con el formato `atus_anual_AAAA.csv`.

5.2 Pasos de Ejecución

Paso 1 -- Levantar el cluster

```
cd atus_project/docker
docker compose up -d ray-head ray-worker-1 ray-worker-2
```

Esperar aproximadamente 30 segundos hasta que Ray este completamente inicializado.

Paso 2 -- Ejecutar el ETL

```
docker compose --profile etl up etl
```

Convierte todos los CSV a Parquet en `data/parquet/`. El proceso puede tardar varios minutos según el número de archivos.

Paso 3 -- Ejecutar el análisis

```
docker compose --profile analisis up analisis
```

Genera `data/reports/resultados.json` con todos los indicadores calculados de forma distribuida.

Paso 4 -- Acceder al dashboard

Abrir en el navegador: `http://localhost:8501`

(Opcional) Paso 5 -- Benchmark Ray vs Pandas

```
docker exec atus-ray-head python src/analisis.py benchmark
```

Genera `data/reports/benchmark.json` y lo muestra en la pestaña Benchmark del dashboard.

5.3 URLs de Acceso

Servicio	URL
Dashboard ATUS (Streamlit)	<code>http://localhost:8501</code>
Ray Dashboard	<code>http://localhost:8265</code>

5.4 Escalado de Workers

Para agregar workers adicionales se puede usar el flag `--scale` de Docker Compose:

```
docker compose up -d --scale ray-worker-1=4
```

Alternativamente, duplicar el bloque de configuración de `ray-worker-2` en `docker-compose.yml` con un nombre distinto (`ray-worker-3`, etc.).

6. Dependencias y Tecnologías

Libreria / Herramienta	Version	Rol en el proyecto
Python	3.11	Lenguaje principal
Ray	2.10.0	Motor de procesamiento distribuido
Pandas	2.2.2	ETL y analisis de datos
PyArrow	15.0.2	Lectura y escritura de Parquet
fastparquet	2024.2.0	Backend alternativo para Parquet
Streamlit	1.33.0	Dashboard web interactivo
Plotly	5.21.0	Visualizaciones graficas interactivas
openpyxl	3.1.2	Soporte de formatos Excel
Docker Compose	v2	Orquestacion de contenedores

7. Consideraciones de Diseño

7.1 Encoding de los Datos INEGI

Los archivos CSV del ATUS estan codificados en latin-1 (ISO-8859-1), estandar historico del INEGI. El ETL siempre intenta la lectura con este encoding en primer lugar y recurre a UTF-8 como fallback. Adicionalmente, la funcion `_fix_encoding` aplica una correccion de doble decodificacion para cadenas que hayan sido mal leidas como UTF-8 cuando en realidad eran latin-1, seguida de normalizacion NFC con `unicodedata`.

7.2 Estrategia ETL vs. Analisis

El ETL se ejecuta de forma secuencial en el head node porque los archivos CSV raw solo estan montados en ese nodo (volumen de solo lectura). El procesamiento distribuido comienza en la fase de analisis, donde los Parquet ya estan disponibles en el volumen compartido `atus-data` y cualquier worker puede leerlos directamente.

7.3 Formato Parquet con Snappy

Se eligio Apache Parquet con compresion Snappy como formato intermedio por su reduccion significativa de tamano respecto al CSV original, su lectura columnar eficiente para las agregaciones del analisis y su compatibilidad nativa con Pandas y PyArrow en el entorno distribuido.

7.4 Indice de Gravedad

El campo GRAVEDAD es una metrica sintetica definida como $TOTAL_MUERTOS \times 3 + TOTAL_HERIDOS$. El factor multiplicador de 3 asigna mayor peso relativo a los fallecimientos respecto a los lesionados, permitiendo comparar la severidad ponderada de accidentes entre estados, municipios y periodos.

7.5 Gestion de Memoria en Ray

Para reducir el uso de memoria en los workers, cada tarea remota recibe unicamente las columnas que necesita. El DataFrame completo se sube una sola vez al object store de Ray mediante `ray.put()` y cada tarea recibe una referencia (`ObjectRef`) en lugar de una copia completa del DataFrame.

Anexo — Imágenes del Sistema en Funcionamiento

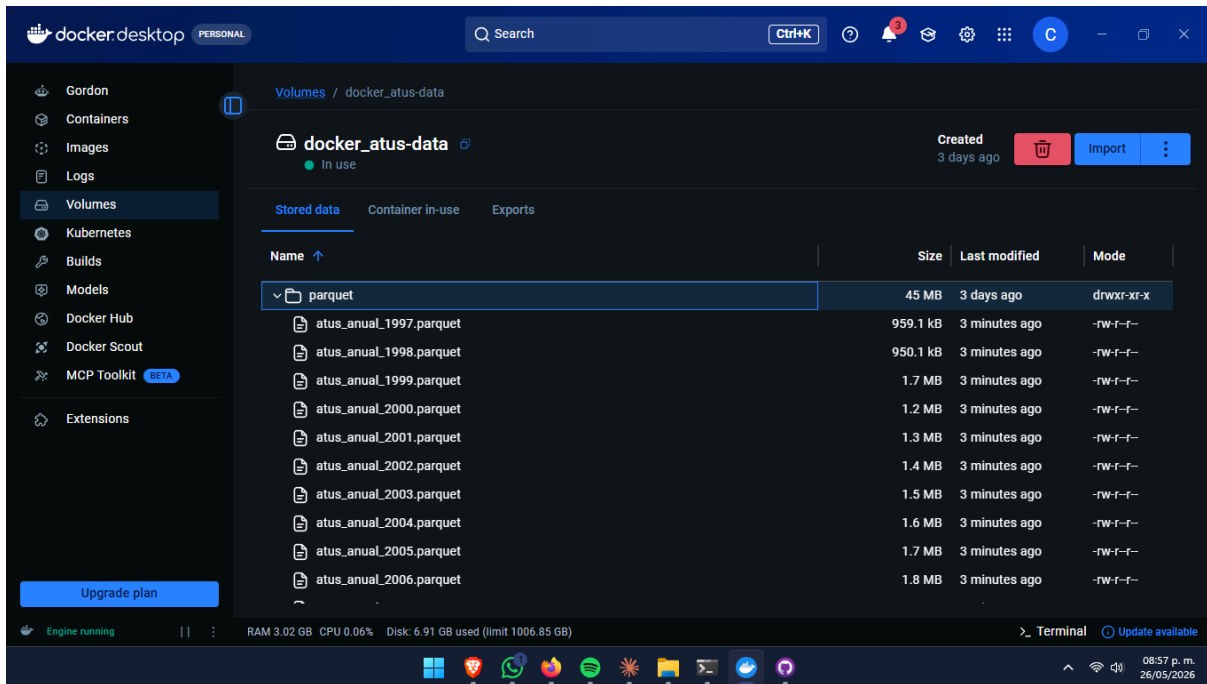
En este apartado se reservan los espacios para las capturas de pantalla que documentan el funcionamiento del sistema. Insertar las imágenes correspondientes en cada recuadro.

A.1 Pipeline ETL

```
PS C:\Escuela\Pruebas de códigos\Python\ComputoParalelo_ProyectoFinal\atus_project\docker> docker exec atus-ray-head python3 src/etl.py

=====
ETL ATUS - modo: SECUENCIAL (head)
raw_dir      : /app/data/raw
parquet_dir  : /app/data/parquet
=====

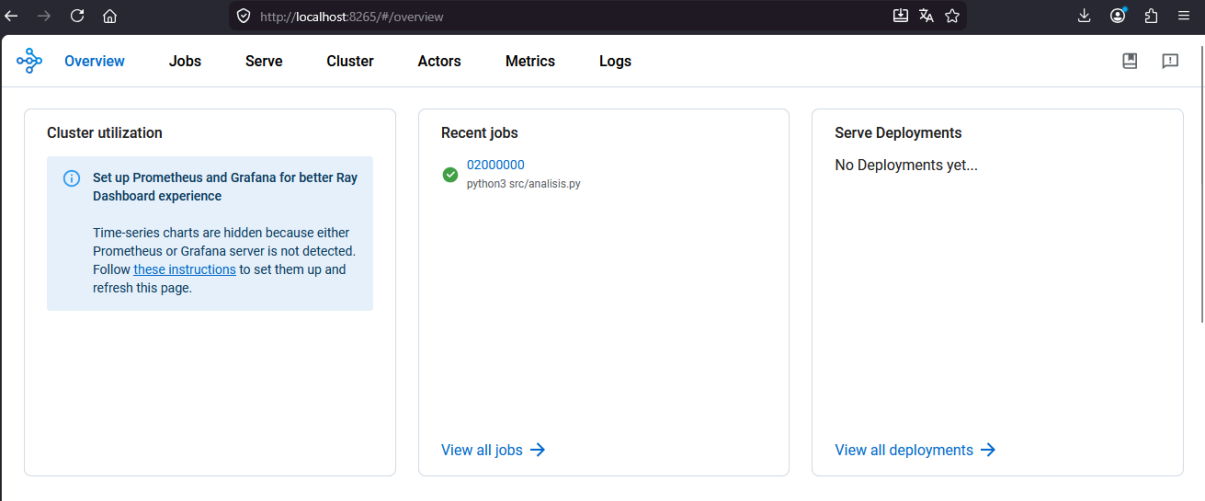
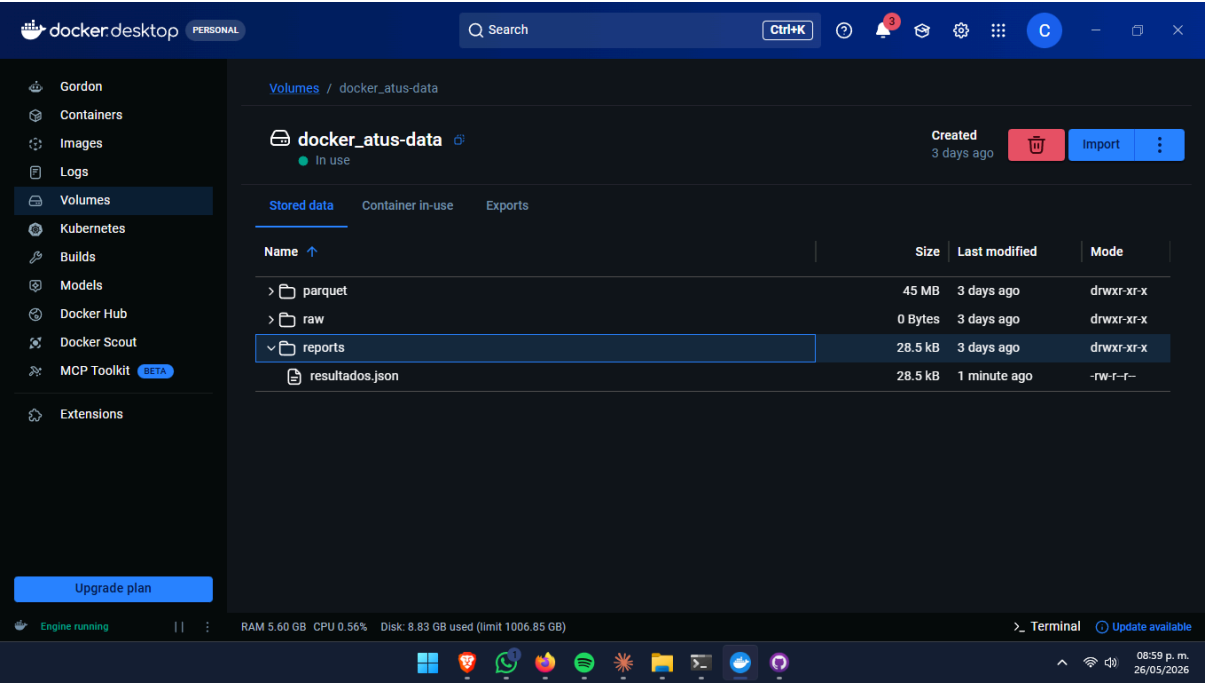
Archivos procesados : 28
Registros totales   : 6,828,654
Tiempo              : 78.76s
```



A.2 Análisis Distribuido con Ray

```
PS C:\Escuela\Pruebas de códigos\Python\ComputoParalelo_ProyectoFinal\atus_project\docker> docker exec atus-ray-head python3 src/analisis.py
2026-05-27 02:58:11,613 INFO worker.py:1567 -- Connecting to existing Ray cluster at address: ray-head:6379...
2026-05-27 02:58:11,621 INFO worker.py:1743 -- Connected to Ray cluster. View the dashboard at http://172.18.0.2:8265
Cargando datos...
6,828,654 registros cargados (4.0s)
Lanzando análisis distribuido...
Análisis completado en 2.59s

Resultados guardados en: /app/data/reports/resultados.json
PS C:\Escuela\Pruebas de códigos\Python\ComputoParalelo_ProyectoFinal\atus_project\docker> |
```



Cluster status and autoscaler

Node count

Set up Prometheus and Grafana for better Ray Dashboard experience

Time-series charts are hidden because either Prometheus or Grafana server is not detected. Follow [these instructions](#) to set them up and refresh this page.

[View all nodes](#)

Node Status

Active:

- 1 node_c5e0a1a8190a61d53e5b4d450bbd2c12319697ac
- 1 node_13cb36bb8ae229ca6400398f98e6fc595d40b2673!
- 1 node_b15ad15076b73692e6e3e8bddf66531652e98a6e3

Pending:
(no pending nodes)

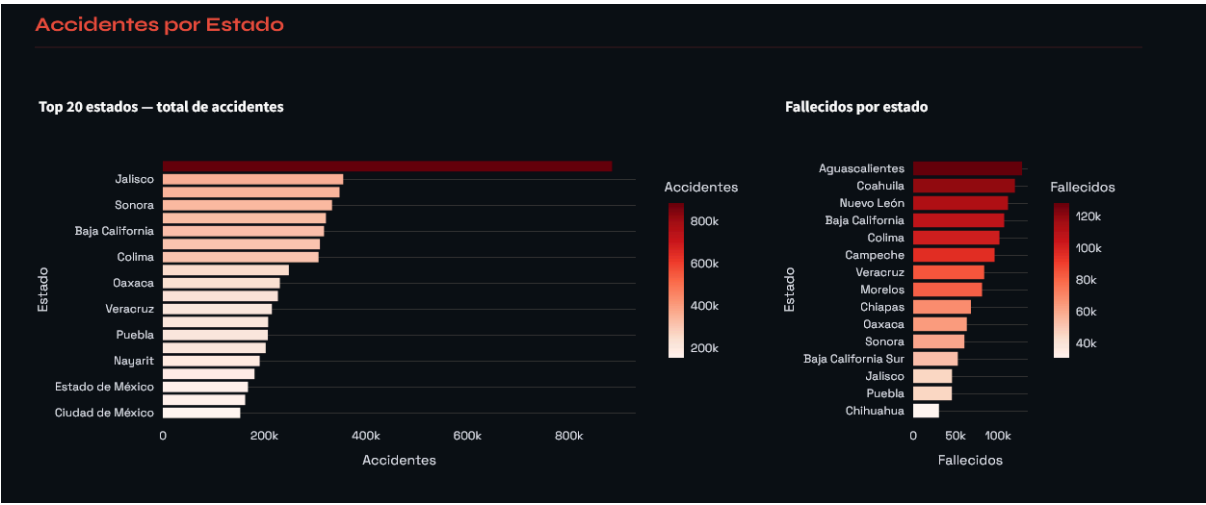
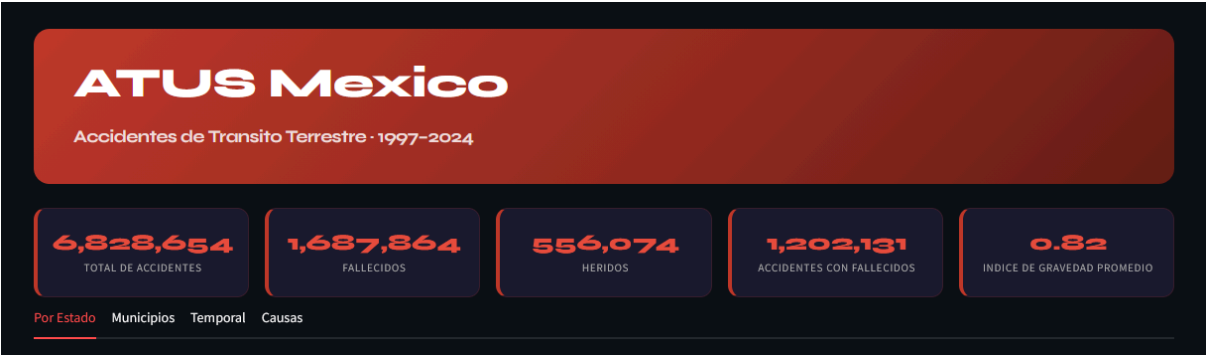
Recent failures:
(no failures)

Resource Status

Usage:
0.0/6.0 CPU
0B/14.45GiB memory
0B/6.83GiB object_store_memory

Demands:
(no resource demands)

A.3 Dashboard Streamlit



Estado	Accidentes	Fallecidos	Heridos	Gravedad
Nuevo León	884,533	111,512	47,470	382,006
Jalisco	355,193	45,672	23,013	160,029
Coahuila	347,911	119,655	26,313	385,278
Sonora	333,141	60,223	26,866	207,535
Campeche	321,149	95,734	7,555	294,757
Baja California	317,361	107,014	21,006	342,048
Aguascalientes	309,261	128,053	11,858	396,017
Colima	306,567	101,464	11,080	315,472
Morelos	248,124	81,016	13,128	256,176
Oaxaca	230,440	63,102	9,218	198,524

